



中国海洋大学
OCEAN UNIVERSITY OF CHINA

系统开发工具基础实验报告

第 2 周：命令行环境；开发环境与工具；Debugging and Profiling

姓名： 姬艺华

学号： 25020007046

日期： 2026 年 8 月 31 日

目录

1	课上习题	3
1.1	第 5 题：控制一个可清理的后台任务	3
1.1.1	任务说明	3
1.1.2	关键操作	3
1.1.3	实验结果	4
1.2	第 6 题：语义重构与本地开发反馈	4
1.2.1	任务说明	4
1.2.2	关键操作	4
1.2.3	实验结果	5
1.3	第 7 题：用调试器定位归并排序缺陷	7
1.3.1	任务说明	7
1.3.2	关键操作	7
1.3.3	实验结果	9
1.4	第 8 题：先测量，再优化慢速词频程序	11
1.4.1	任务说明	11
1.4.2	关键操作	11
1.4.3	实验结果	12
2	课下习题	14
2.1	启用 Vim 模式	14
2.2	完成 VimGolf 挑战	14
2.3	配置 IDE 扩展和语言服务器	15
2.4	安装新的 IDE 扩展	16
2.5	-- 参数作用	16
2.6	ls 参数组合	17
2.7	进程替换	17
2.8	marco 与 polo 函数	18
2.9	循环执行程序直到失败	19
2.10	目录文件数量统计脚本	22
2.11	文本文件单词数量统计脚本	23
3	Git 与作业提交	24
3.1	仓库内容整理	24
3.2	GitHub / Gitee 提交记录	24
3.3	commit 记录截图、仓库页面截图	24

4 问题与总结	25
4.1 遇到的问题	25
4.2 实验总结	25

1 课上习题

本模块记录上课实验四道例题操作基本过程，以及关键点。

1.1 第 5 题：控制一个可清理的后台任务

主题：命令行环境：进程、信号与任务控制

1.1.1 任务说明

运行 worker.sh 脚本，重定向标准输出与错误输出；Ctrl+Z 挂起进程，bg 放到后台；通过 jobs -p 获取 PID，发送 SIGTERM 终止任务，等待进程结束，确认 cleanup.log 输出 CLEAN_EXIT。

1.1.2 关键操作

将脚本执行，并将 stdout 和 stderr 写入 stdout.log、stderr.log；将任务挂起，并让它在后台继续运行；使用 jobs 确认状态。

```
./worker.sh > stdout.log 2> stderr.log  
(ctrl+z)  
bg  
jobs
```

使用 jobs-p 获取 PID；发送 SIGTERM 结束任务；等待任务结束后，确认 cleanup.log 中出现 CLEAN_EXIT。

```
pid=$(jobs -p)  
echo "$pid"  
kill -TERM "$pid"  
wait "$pid"  
cat cleanup.log
```

关键点：

- Ctrl+Z 暂停正在运行的进程；bg 将挂起的任务放到后台继续运行；jobs 查看当前终端的后台任务。
- PID 代表进程编号，用来定位要操作的进程；jobs -p 获取后台任务对应的 PID。
- pid=\$(jobs -p)：\$() 执行括号内命令，将命令输出保存到 pid 变量。
- kill -TERM "\$pid"：向对应 PID 进程发送 SIGTERM 信号，允许程序执行清理逻辑；不要使用 kill -9，强制杀死不会执行清理代码。
- wait "\$pid"：等待该进程完全结束，进程退出完成后再读取日志。
- cat cleanup.log 查看输出日志，出现 CLEAN_EXIT 代表脚本完成清理后退出。
- Ctrl+Z、bg、jobs 仅交互式终端可用，写到 shell 脚本中无法正常获取后台作业

1.1.3 实验结果

```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.31/q05
$ ./worker.sh > stdout.log 2> stderr.log

[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.31/q05
$ bg
[1]+  ./worker.sh > stdout.log 2> stderr.log &

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.31/q05
$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &
```

图 1 确认后台运行情况

```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.31/q05
$ kill -TERM "$pid"

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.31/q05
$ wait "$pid"
[1]+  Done                    ./worker.sh > stdout.log 2> stderr.log

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.31/q05
$ jobs

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.31/q05
$ cat cleanup.log
CLEAN_EXIT
```

图 2 确认结束任务以及 cleanup.log 文件内容

1.2 第 6 题：语义重构与本地开发反馈

主题：开发环境与工具

1.2.1 任务说明

借助编辑器语言服务器完成代码语义重构；使用 ruff 做代码静态检查，pytest 执行单元测试，保证修改后代码功能正常。

1.2.2 关键操作

检查环境：

```
python --version
python --version
ruff --version
```

在文件 app.py 中加入未使用过的 import os，随后用 ruff 发现并删除；最后运行保证通过。

```
ruff check .
pytest
```

关键点：

- 语言服务器：提供跳转定义、查找引用等代码分析功能。
- 重命名符号：语义层面修改名称，同步更新全部调用位置，不是简单文本替换。
- ruff：静态检查工具，检测未使用导入、格式类代码问题。
- pytest 单元测试：`test_` 开头函数作为测试用例，用 `assert` 校验结果。
- 修改代码后依次运行 ruff、pytest，确认代码规范与功能没有问题。

1.2.3 实验结果

分别演示跳转到定义和查找引用：

```
PS D:\classPai\Class\8.24\q06> python --version
Python 3.9.12
PS D:\classPai\Class\8.24\q06> pytest --version
pytest 8.4.2
PS D:\classPai\Class\8.24\q06> ruff --version
ruff 0.4.10
PS D:\classPai\Class\8.24\q06> python app.py
PS D:\classPai\Class\8.24\q06> python app.py
50.0
```

图 3 环境检查，程序试运行

```
1 from math_utils import total_price
2 print(total_price(12.5, 4))
```

CodeGeeX

转到定义 F12

转到声明

转到类型定义

图 4 右键显示信息

```
q06 > math_utils.py > total_price
解释 | 添加注释 | × |
1 def total_price(price: float, count: int) -> float:
2     return price * count
```

图 5 转到定义



图 6 右键显示信息

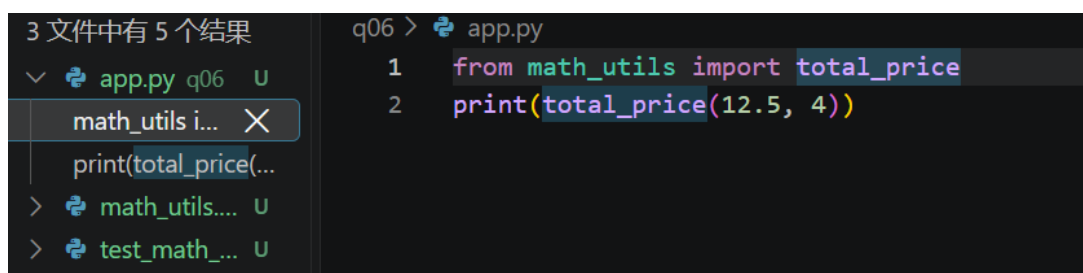


图 7 查找所有引用

使用“查找所有引用”后,编辑器能够找到 `app.py` 和 `test_math_utils.py` 中对 `total_price` 的引用。

鼠标放在 `total_price` 重命名符号, 输入 `calculate_total`:

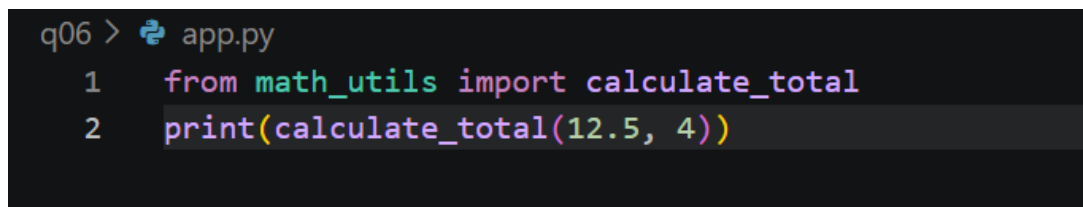


图 8

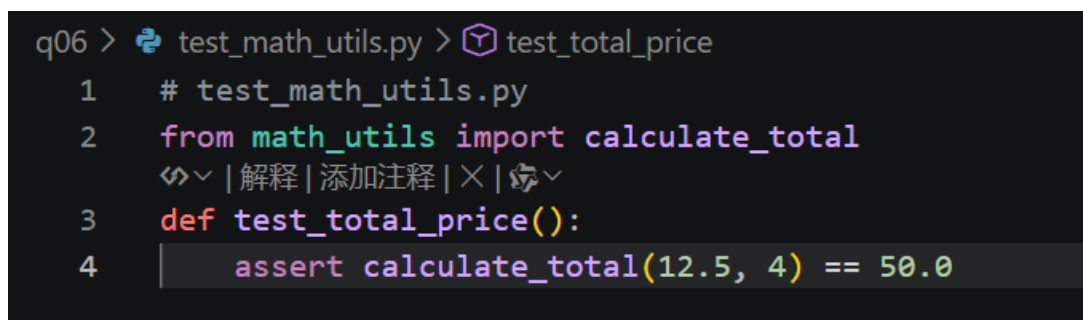


图 9

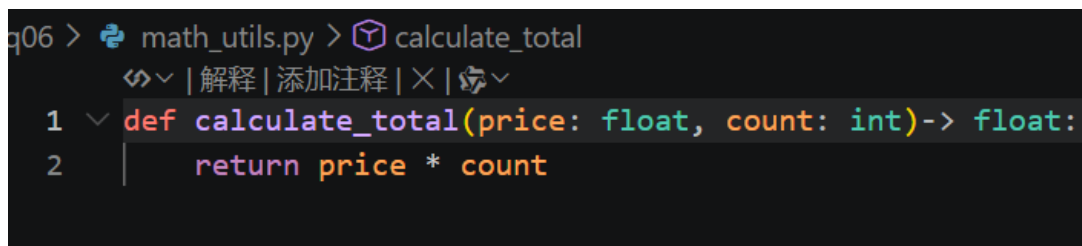


图 10

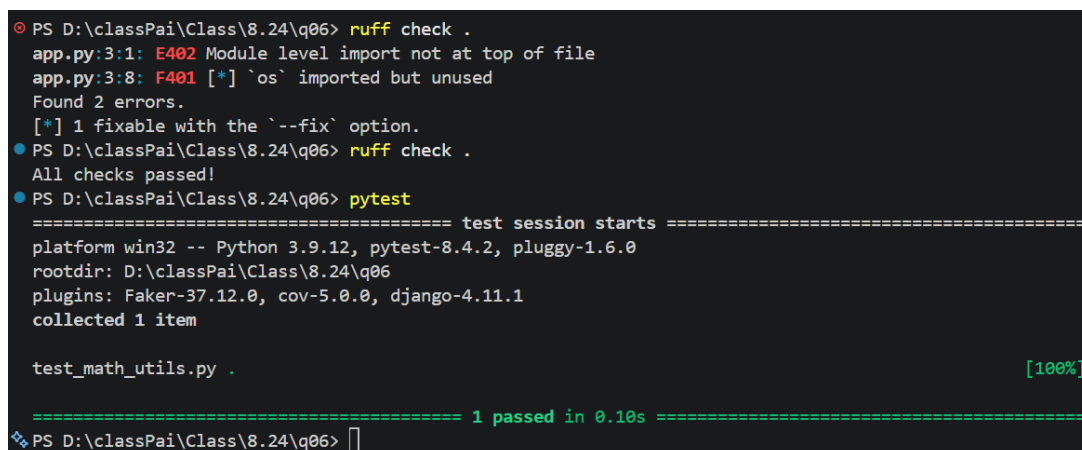


图 11 最后运行 ruff 和 pytest 均通过

1.3 第 7 题：用调试器定位归并排序缺陷

主题：Debugging

1.3.1 任务说明

调试存在缺陷的归并排序代码，设置断点观察变量定位下标错误，完成最小修复，编写 pytest 用例验证修复结果。

1.3.2 关键操作

先运行输出查看，发现结果不正确；使用 IDE debugger 调试，在 `while i < len(left) and j < len(right)` 处；设置监视变量；使用 F10 单步执行。

观察 `i`、`j`、`left[i]` 和 `right[j]`；当 `left=[1,3]`、`right=[1,4]`、`i=1`、`j=0` 时，`left[i]=3`，`right[j]=1`，程序进入 `else` 分支。此时应将 `right[j]` 即 1 加入 `result`，但原代码使用了 `right[i]`，实际加入 `right[1]=4`，导致归并结果错误。

此时需要修复，改为：

```
1 def merge(left, right):
2     result = []
3     i = j = 0
4     while i < len(left) and j < len(right):
5         if left[i] <= right[j]:
6             result.append(left[i]); i += 1
7         else:
```



```
8         result.append(right[j]); j += 1 # 此处存在缺陷
9     return result + left[i:] + right[j:]
10 def merge_sort(a):
11     if len(a) <= 1: return a
12     m = len(a) // 2
13     return merge(merge_sort(a[:m]), merge_sort(a[m:]))
14
15 print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
```

使用 pytest 增加两个测试：给定输入和含重复元素的列表；保证测试通过。

```
1 #test_merge_sort.py
2 from merge_sort import merge_sort
3 def test_given_input():
4     assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]
5 def test_duplicate_elements():
6     assert merge_sort([2, 2, 1, 3, 1]) == [1, 1, 2, 2, 3]
```

解释代码：

- `from merge_sort import merge_sort`：导入归并排序函数，测试文件才能调用它。
- 函数名以 `test_` 开头，pytest 才会识别为测试用例。
- `assert` 实际结果 == 预期结果：两边相等测试通过，不相等则报错。
- `test_given_input`：题目给定输入；`test_duplicate_elements`：包含重复元素的列表。

关键点：

- 断点：调试器在此暂停，查看变量实时运行值。
- `i`、`j` 分别对应左右子数组下标，二者不能混用。
- 原始 bug：错误写 `right[i]`，正确写法为 `right[j]`。

1.3.3 实验结果

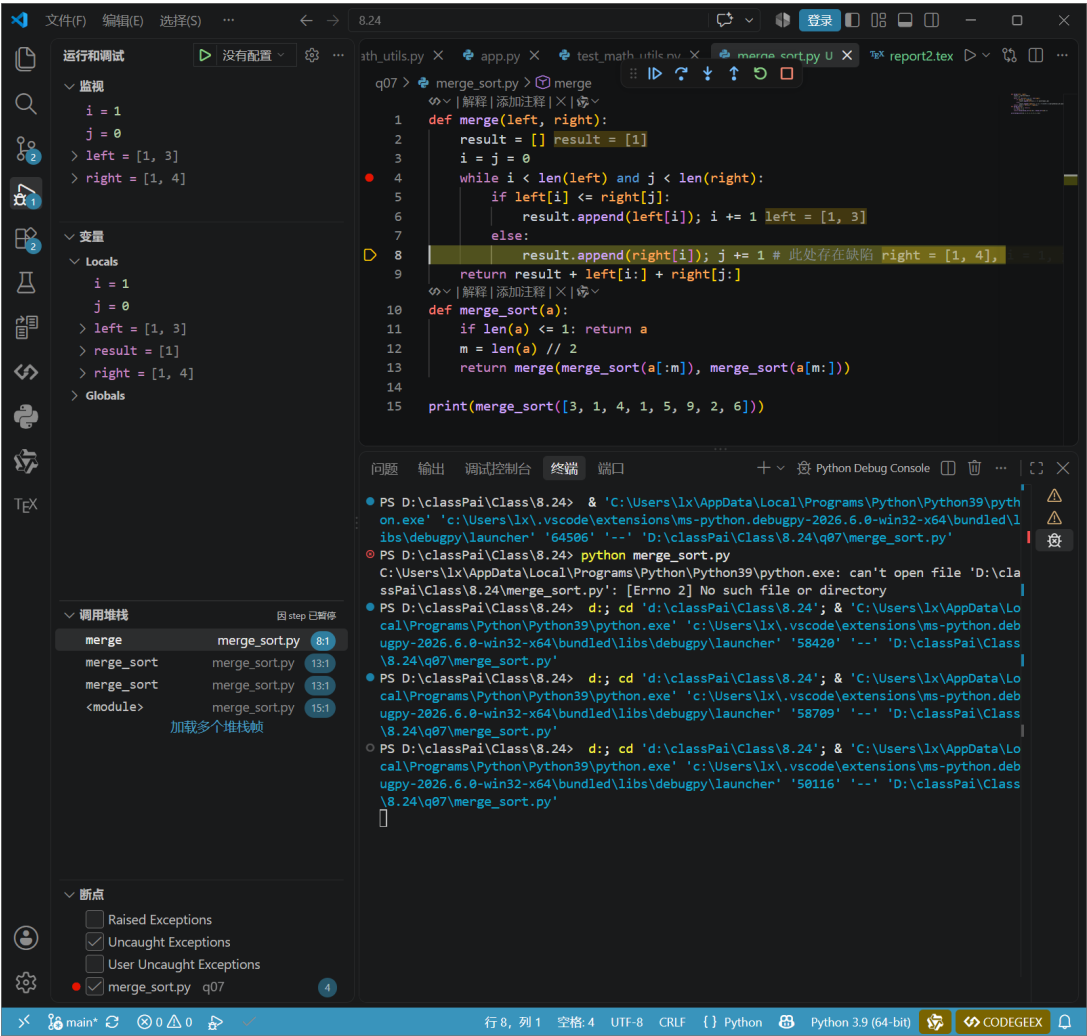


图 12 设置断点

```
q07 > merge_sort.py > ...
  1 def merge(left, right):
  2     result = []
  3     i = j = 0
  4     while i < len(left) and j < len(right):
  5         if left[i] <= right[j]:
  6             result.append(left[i]); i += 1
  7         else:
  8             result.append(right[j]); j += 1 # 此处存在缺陷
  9     return result + left[i:] + right[j:]
 10
 11 def merge_sort(a):
 12     if len(a) <= 1: return a
 13     m = len(a) // 2
 14     return merge(merge_sort(a[:m]), merge_sort(a[m:]))
 15
 16 print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
```

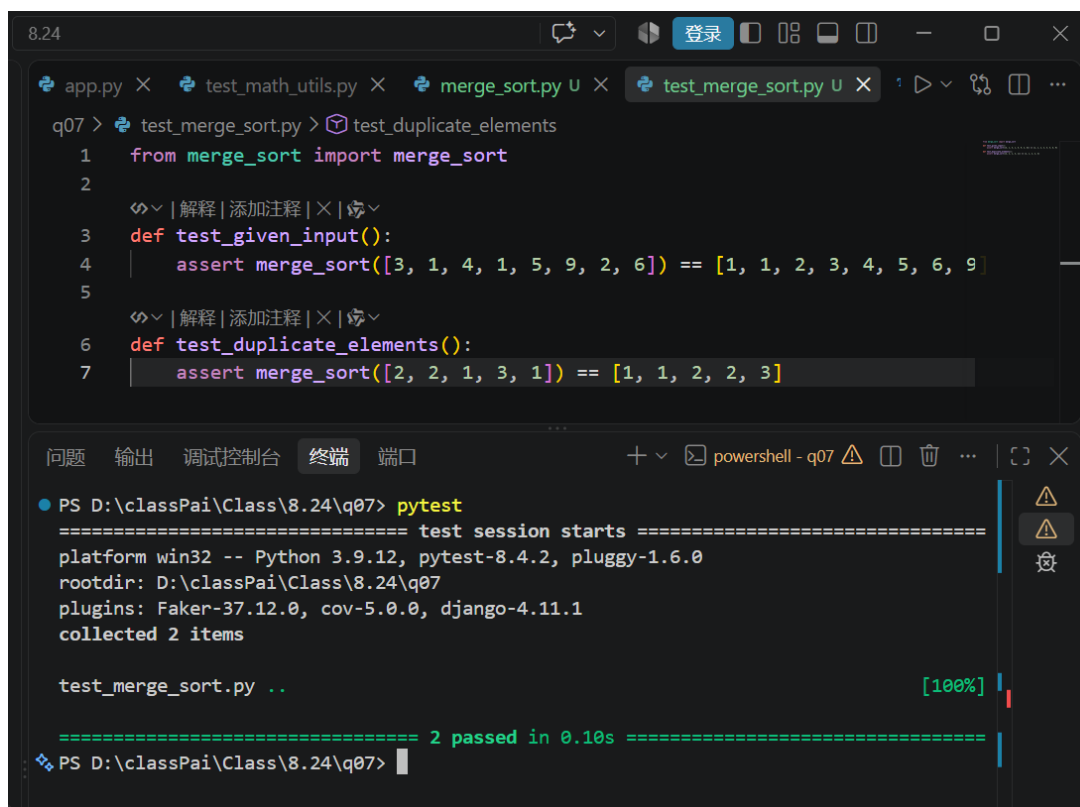
问题 输出 调试控制台 终端 端口 + powershell - q07

```
PS D:\classPai\Class\8.24\q06> pytest
collected 1 item

test_math_utils.py .
[100%]

===== 1 passed in 0.10s =====
=====
PS D:\classPai\Class\8.24\q06> cd ..
PS D:\classPai\Class\8.24> cd q07
PS D:\classPai\Class\8.24\q07> python merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
PS D:\classPai\Class\8.24\q07> python merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
PS D:\classPai\Class\8.24\q07>
```

图 13 完成最小修复



```
8.24
app.py × test_math_utils.py × merge_sort.py U × test_merge_sort.py U ×
q07 > test_merge_sort.py > test_duplicate_elements
1 from merge_sort import merge_sort
2
3 def test_given_input():
4     assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]
5
6 def test_duplicate_elements():
7     assert merge_sort([2, 2, 1, 3, 1]) == [1, 1, 2, 2, 3]

问题 输出 调试控制台 终端 端口
PS D:\classPai\Class\8.24\q07> pytest
===== test session starts =====
platform win32 -- Python 3.9.12, pytest-8.4.2, pluggy-1.6.0
rootdir: D:\classPai\Class\8.24\q07
plugins: Faker-37.12.0, cov-5.0.0, django-4.11.1
collected 2 items

test_merge_sort.py .. [100%]

===== 2 passed in 0.10s =====
PS D:\classPai\Class\8.24\q07>
```

图 14 测试通过

1.4 第 8 题：先测量，再优化慢速词频程序

主题：Profiling

1.4.1 任务说明

使用 time、cProfile 对词频统计程序做性能分析；找到性能瓶颈进行代码优化，记录耗时，计算优化前后的加速比。

1.4.2 关键操作

生成 words.txt 使用 time 运行原程序 2 次并记录耗时：

```
python generate_words.py
ls
python wordfreq.py
time python wordfreq.py
time python wordfreq.py
```

使用 cProfile-s cumulative 运行一次，找出主要耗时位置；优化程序；将 list 改为“去重”的 set：

```
python -m cProfile -s cumulative wordfreq.py
```

优化后 wordfreq.py:

```
1 words = open("words.txt", encoding="utf-8").read().split()
2 unique = set()
3
4 for word in words:
5     unique.add(word)
6
7 print("count=", len(unique))
```

关键点:

- `time`: 用来测量程序实际运行耗时。
- `cProfile` 性能剖析工具, 按累计耗时定位性能瓶颈。
- `list` 成员查找效率低, 改用 `set` 集合实现去重提升速度。
- 优化不能改变程序输出结果; 多次运行记录耗时, 计算加速比。

1.4.3 实验结果

优化前后耗时对比:

	第 1 次	第 2 次	中位数
优化前 (list)	0.205s	0.191s	0.198s
优化后 (set)	0.131s	0.106s	0.1185s

表 1 优化前后运行耗时对比

加速比计算:

$$\text{加速比} = \frac{\text{优化前中位数}}{\text{优化后中位数}} = \frac{0.198}{0.1185} \approx 1.67 \tag{1}$$

将 `list` 替换为 `set` 后, 去重操作的时间复杂度降低, 程序运行速度提升约 **1.67** 倍。实际耗时还受文件读取、Python 启动等因素影响, 但集合在大数据量下扩展性更好。

```

● PS D:\classPai\Class\8.24\q08> python generate_words.py
● PS D:\classPai\Class\8.24\q08> ls

目录: D:\classPai\Class\8.24\q08

Mode                LastWriteTime         Length Name
----                -
-a----            2026/8/31      17:20          200 generate_words.py
-a----            2026/9/1       10:38          178 wordfreq.py
-a----            2026/9/1       10:41       146755 words.txt

● PS D:\classPai\Class\8.24\q08> python wordfreq.py
count= 1000

```

图 15 原程序运行

```

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m0.205s
user    0m0.031s
sys     0m0.015s

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m0.191s
user    0m0.000s
sys     0m0.015s

```

图 16 使用 time 运行原程序

```

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q08 (main)
$ python -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.088 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
1      0.000    0.000    0.088    0.088    {built-in method builtins.exec}
1      0.086    0.086    0.088    0.088    wordfreq.py:1(<module>)
1      0.001    0.001    0.001    0.001    {method 'split' of 'str' objects}
1      0.000    0.000    0.000    0.000    {built-in method builtins.print}
1      0.000    0.000    0.000    0.000    {method 'read' of '_io.TextIOWrapper' objects}
1      0.000    0.000    0.000    0.000    {built-in method io.open}
1      0.000    0.000    0.000    0.000    codecs.py:319(decode)
1      0.000    0.000    0.000    0.000    {built-in method _codecs.utf_8_decode}
1000    0.000    0.000    0.000    0.000    {method 'append' of 'list' objects}
1      0.000    0.000    0.000    0.000    codecs.py:309(__init__)
1      0.000    0.000    0.000    0.000    codecs.py:260(__init__)
1      0.000    0.000    0.000    0.000    {method 'disable' of '_lsprof.Profiler' objects}
1      0.000    0.000    0.000    0.000    {built-in method builtins.len}

```

图 17 找出主要耗时位置

```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m0.118s
user    0m0.015s
sys     0m0.000s

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24/q08 (main)
$ time python wordfreq.py
count= 1000

real    0m0.102s
user    0m0.031s
sys     0m0.000s
```

图 18 再次使用 time 计算运行时间

2 课下习题

本部分完成 Vim 模式、VimGolf、IDE 语言服务器和编辑器扩展相关练习，主要熟悉文本编辑效率和开发环境配置。

2.1 启用 Vim 模式

在 VS Code 中安装 Vim 扩展，并在 Git Bash 中启用 Vi 编辑模式。

```
set -o vi
set -o | grep vi
```

在 VS Code 中可以使用 Vim 的普通模式和插入模式，例如按 `i` 进入插入模式，按 `Esc` 返回普通模式，并使用 `h`、`j`、`k`、`l` 移动光标。

```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24 (main)
$ set -o vi

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8.24 (main)
$ set -o | grep vi
privileged    off
vi            on
```

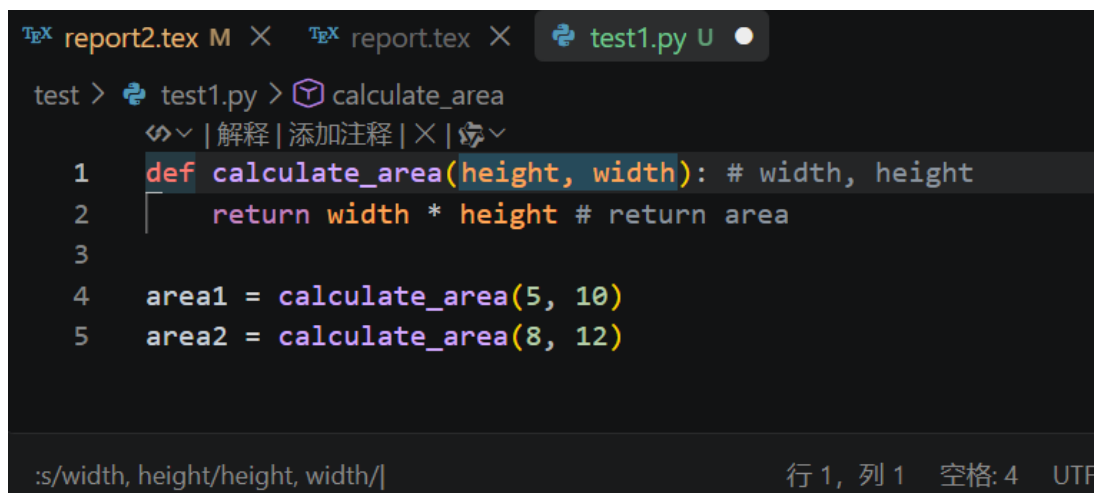
图 19 在 Shell 中启用 Vim 模式

2.2 完成 VimGolf 挑战

选择了一道交换 Python 函数参数并删除注释的 VimGolf 练习。主要使用以下 Vim 命令：

```
ggdd
:s/width, height/height, width/
:%s/ *#.*//
```

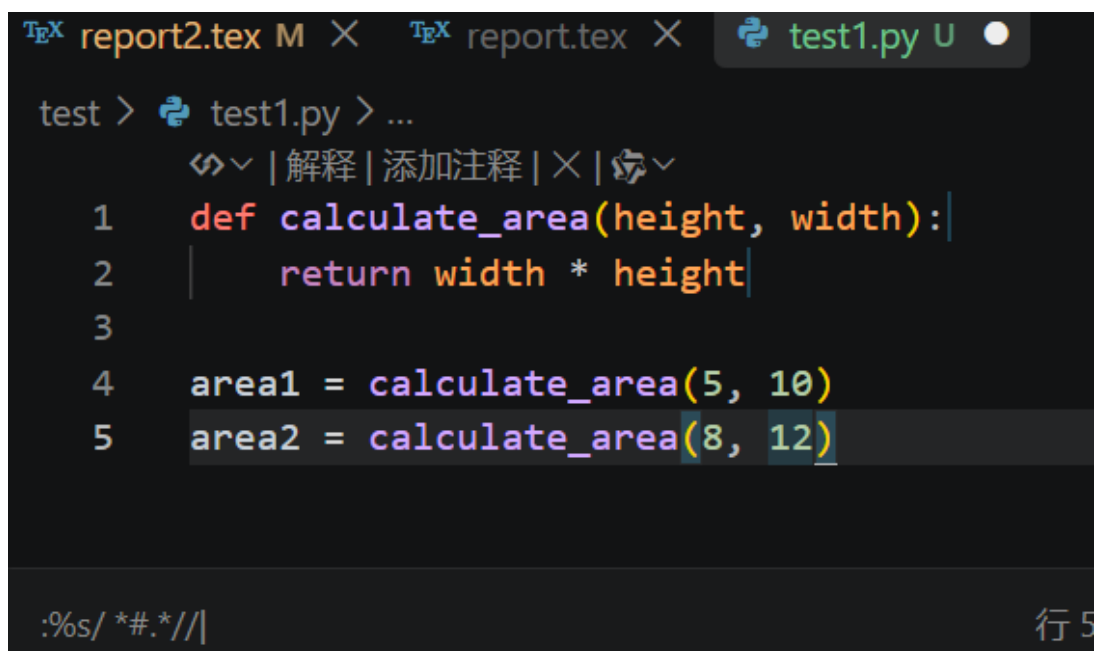
其中, `ggdd` 用于删除第一行, `s` 用于替换当前行内容, `%s` 用于在整个文件中进行替换。最终成功将函数参数交换, 并删除全部注释。



```
test > test1.py > calculate_area
| 解释 | 添加注释 | × |
1 def calculate_area(height, width): # width, height
2     return width * height # return area
3
4 area1 = calculate_area(5, 10)
5 area2 = calculate_area(8, 12)

:s/width, height/height, width/| 行 1, 列 1 空格: 4 UTF
```

图 20 完成 VimGolf 挑战



```
test > test1.py > ...
| 解释 | 添加注释 | × |
1 def calculate_area(height, width):|
2     return width * height|
3
4 area1 = calculate_area(5, 10)
5 area2 = calculate_area(8, 12)

:%s/ *#.*/| 行 5
```

图 21 完成 VimGolf 挑战

2.3 配置 IDE 扩展和语言服务器

使用已有的 Python 项目进行测试, 在 VS Code 中启用 Python 和 Pylance 扩展, 并对函数进行跳转到定义测试。

在 `app.py` 中将光标放在 `calculate_total` 上, 使用 `F12` 后可以正确跳转到 `math_utils.py` 中对应的函数定义, 说明语言服务器工作正常。

```
1 from math_utils import calculate_total
2
3 print(calculate_total(12.5, 4))
```

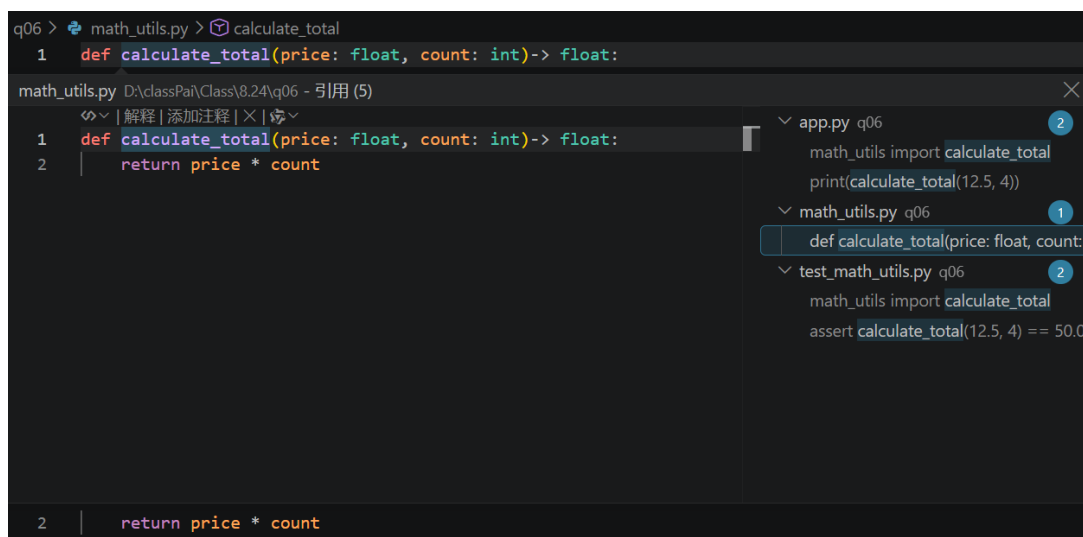



图 22 使用语言服务器跳转到函数定义

2.4 安装新的 IDE 扩展

浏览 VS Code 扩展列表后安装了 Error Lens 扩展。为了测试扩展功能，在 Python 文件中临时加入一个未定义变量：

```
1 print(abc)
```

Error Lens 可以直接在代码旁显示“未定义 abc”的错误信息，使代码问题更加直观。测试完成后删除该错误代码。

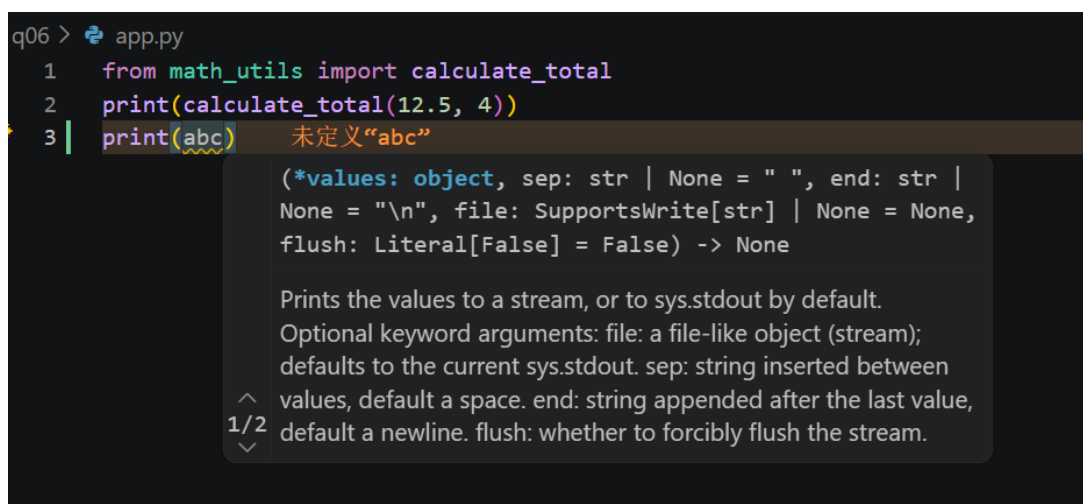


图 23 Error Lens 显示代码错误

2.5 -- 参数作用

```
touch -- -myfile
rm ./-myfile
```

使用 -- 创建以短横线开头的文件名，避免被命令解析为参数选项。删除该文件时使用 ./-myfile 指定路径，避免 rm 将其识别为选项。

```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8031
$ touch -- -myfile

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8031
$ ls
-m myfile  8.31.docx

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8031
$ rm -myfile
rm: unknown option -- m
Try 'rm ./-myfile' to remove the file '-myfile'.
Try 'rm --help' for more information.

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8031
$ rm ./-myfile

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8031
$ ls
8.31.docx
```

图 24 使用 -- 创建和删除特殊文件名

2.6 ls 参数组合

```
ls -laht --color=auto
```

该命令实现显示所有文件（包括隐藏文件）、显示详细信息、以易读格式显示文件大小、按照修改时间排序，并开启颜色显示。

```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8031
$ ls -laht --color=auto
total 1.1M
drwxr-xr-x 1 1x 197121  0 Sep  3 13:05 ./
-rw-r--r-- 1 1x 197121 1.1M Sep  2 11:53 8.31.docx
drwxr-xr-x 1 1x 197121  0 Sep  1 12:15 ../
```

图 25 使用 ls 参数查看文件信息

2.7 进程替换

```
diff <(printenv | sort) <(export | sort)
```

使用进程替换将命令输出作为临时文件交给 diff 比较。实验发现 printenv 和 export 的输出格式不同，其中 export 会显示 Shell 变量声明信息。

```

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8031
$ diff <(printenv | sort) <(export | sort)
1,85c1,83
< !::=: \
< ACLOCAL_PATH=/mingw64/share/aclocal:/usr/share/aclocal
< ALLUSERSPROFILE=C:\ProgramData
< ANDROID_AVD_HOME=D:\development\Android\avd\.android\avd
< ANDROID_HOME=D:\development\Android\sdk
< ANDROID_SDK_HOME=D:\development\Android\avd
< APPDATA=C:\Users\lx\AppData\Roaming
< COMMONPROGRAMFILES=C:\Program Files\Common Files
< COMPUTERNAME=LAPTOP-4K9EVO2A
< COMSPEC=C:\windows\system32\cmd.exe
< CONFIG_SITE=/etc/config.site
< CommonProgramFiles(x86)=C:\Program Files (x86)\Common Files
< CommonProgramW6432=C:\Program Files\Common Files
< DISPLAY=needs-to-be-defined
< DriverData=C:\Windows\System32\Drivers\DriverData
< EFC_25128_1592913036=1
< EFC_25128_4126798990=1
< EXEPATH=D:\APPdownload\github_info\Git
< GRADLE_USER_HOME=D:\development\Android\gradle
< HOME=/c/Users/lx
< HOMEDRIVE=C:
< HOMETHROW=Users\lx
< HOSTNAME=LAPTOP-4K9EVO2A
< IGCCSVC_DB=AQAAANCMnd8BFdERjHoAwE/Cl+sBAAAF80bwXoJRuWxoy2dVPBcPQAAAAACAAAAAA
QQZgAAAAEAAACAAABSW/PQEU0n7QHBERh3YtZojXTnGCEDEKuzYrFMR5CfGAAAAAogAAAAAIAACAAAD
mV3V8rA1Cf611Cw8xdMiju1UBpL7zUoXtateJp0so0WAAAADkKyHvRjGzfoaKxGgSfhkEmOrJPnVS3cQ
Tj+axp6pz8C7NPkrfJnuF+LmsmJ2S1a6rQo16Bc/e4o8ILyFXBqaSI9potggBdsapY3r0B0mPYNDRz4M
U7/SNU7KAbn/sgYIAAAAAZEpDKW3Id83hdHISxS/kknFxr1Sb0txyhvPANaxe2NrEODZ9gNcEdToM1rj
gl99vIcVPHGQ2UzyVbjgEI1TijQ==
< INFOPATH=/mingw64/local/info:/mingw64/share/info:/usr/local/info:/usr/share/in
fo:/usr/info:/share/info
< INTEL_DEV_REDIST=C:\Program Files (x86)\Common Files\Intel\Shared Libraries\
< JAVA_HOME=C:\Program Files\Eclipse Adoptium\jdk-17.0.16-hotspot\
< LC_CTYPE=zh_CN.UTF-8
< LOCALAPPDATA=C:\Users\lx\AppData\Local
< LOGONSERVER=\\LAPTOP-4K9EVO2A
< MANPATH=/mingw64/local/man:/mingw64/share/man:/usr/local/man:/usr/share/man:/u
sr/man:/share/man
< MIC_LD_LIBRARY_PATH=C:\Program Files (x86)\Common Files\Intel\Shared Libraries
\compiler\lib\mic
< MINGW_CHOST=x86_64-w64-mingw32
< MINGW_PACKAGE_PREFIX=mingw-w64-x86_64
< MINGW_PREFIX=/mingw64

```

图 26 使用进程替换比较命令输出

2.8 marco 与 polo 函数

```

1 marco() {
2     MARCO_DIR="$PWD"
3 }
4
5 polo() {
6     cd "$MARCO_DIR"
7 }

```

通过 Shell 函数保存当前工作目录。执行 marco 后切换到其他目录，再执行 polo 可以返回之前保存的位置。

```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ touch marco.sh

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ cat > marco.sh
marco() {
    MARCO_DIR="$PWD"
}

polo() {
    cd "$MARCO_DIR"
}

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ source marco.sh

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ marco

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ cd /

1x@LAPTOP-4K9EVO2A MINGW64 /
$ pwd
/

1x@LAPTOP-4K9EVO2A MINGW64 /
$ polo

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ pwd
/d/classPai/Class/8024
```

图 27 测试 marco 和 polo 函数

2.9 循环执行程序直到失败

```
1 while true; do
2     count=$((count + 1))
3
4     ./random.sh > stdout.log 2> stderr.log
5
6     if [[ $? -ne 0 ]]; then
7         echo "Failed after $count runs"
8         cat stdout.log
9         cat stderr.log
10        break
11    fi
12 done
```

编写脚本循环运行随机失败程序，分别保存标准输出和标准错误。当程序返回非零状态时停止运行，输出失败次数和错误信息。本次实验中程序运行 105 次后失败，成功捕获错误输出。

```
lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ touch random.sh

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ cat > random.sh
#!/usr/bin/env bash

n=$(( RANDOM % 100 ))

if [[ n -eq 42 ]]; then
    echo "Something went wrong"
    >&2 echo "The error was using magic numbers"
    exit 1
fi

echo "Everything went according to plan"

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ chmod +x random.sh

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ ls -l
total 1606
-rw-r--r-- 1 lx 197121 1620877 Aug 27 11:58 8.24.docx
-rw-r--r-- 1 lx 197121      65 Sep  3 22:19 marco.sh
drwxr-xr-x 1 lx 197121      0 Aug 26 11:24 q03-git-history-backup/
-rwxr-xr-x 1 lx 197121    205 Sep  3 22:22 random.sh*
-rw-r--r-- 1 lx 197121   15631 Sep  2 11:01 report2.txt
drwxr-xr-x 1 lx 197121      0 Aug 27 23:11 test/

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ touch run_until_fail.sh

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ cat > run_until_fail.sh
#!/usr/bin/env bash

count=0
while true; do
    count=$((count + 1))

    ./random.sh > stdout.log 2> stderr.log

    if [[ $? -ne 0 ]]; then
        echo "Failed after $count runs"
        echo "stdout:"
        cat stdout.log
        echo "stderr:"
        cat stderr.log
        break
    fi
done

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ chmod +x run_until_fail.sh

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ ls -l
total 1607
-rw-r--r-- 1 lx 197121 1620877 Aug 27 11:58 8.24.docx
-rw-r--r-- 1 lx 197121      65 Sep  3 22:19 marco.sh
drwxr-xr-x 1 lx 197121      0 Aug 26 11:24 q03-git-history-backup/
-rwxr-xr-x 1 lx 197121    205 Sep  3 22:22 random.sh*
-rw-r--r-- 1 lx 197121   15631 Sep  2 11:01 report2.txt
-rwxr-xr-x 1 lx 197121    301 Sep  3 22:23 run_until_fail.sh*
drwxr-xr-x 1 lx 197121      0 Aug 27 23:11 test/

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ ./run_until_fail.sh
Failed after 105 runs
stdout:
Something went wrong
```

图 28 循环执行程序直到失败的结果

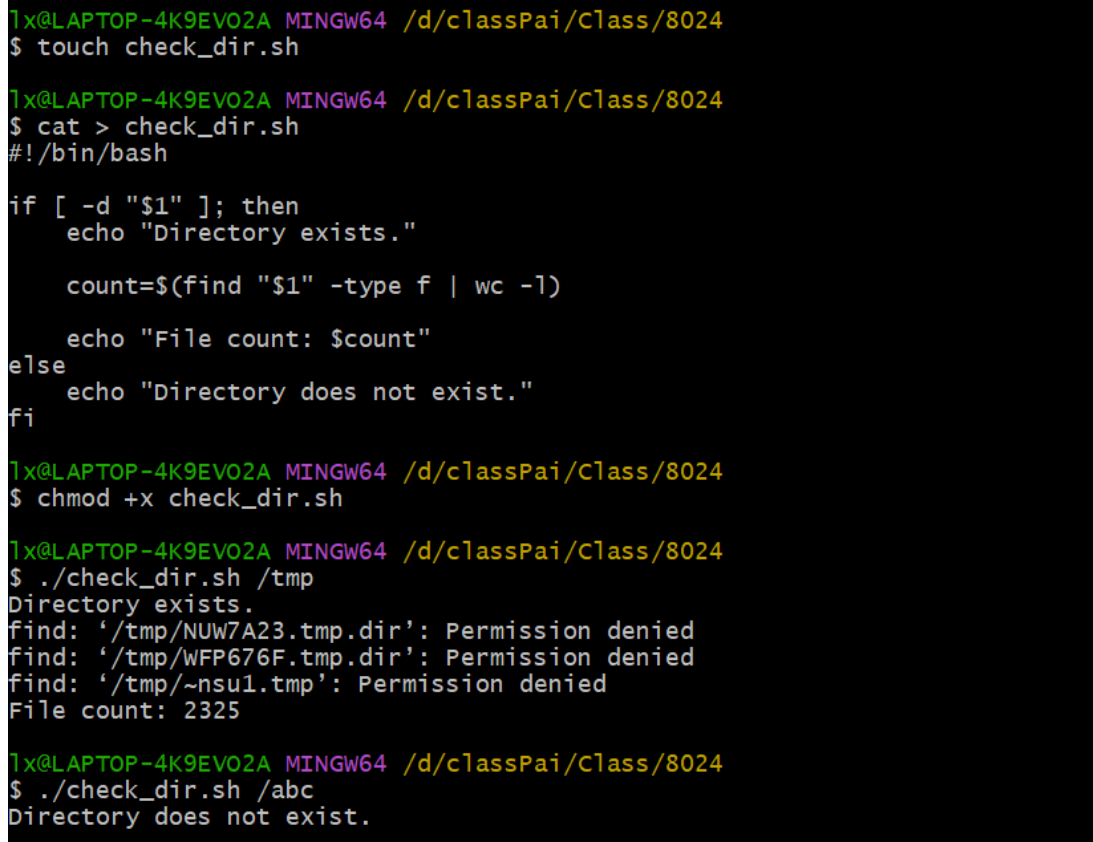
2.10 目录文件数量统计脚本

编写 Shell 脚本 `check_dir.sh`，通过输入目录路径判断目录是否存在，并统计其中普通文件数量。

```
1 #!/bin/bash
2
3 if [ -d "$1" ]; then
4     echo "Directory exists."
5
6     count=$(find "$1" -type f | wc -l)
7
8     echo "File count: $count"
9 else
10     echo "Directory does not exist."
11 fi
```

其中，`$1` 表示脚本输入的的第一个参数，`-d` 用于判断目录是否存在，`find` 命令用于查找普通文件，`wc -l` 用于统计文件数量。

测试结果显示，当输入存在目录时，脚本能够输出文件数量；输入不存在目录时，能够正确提示目录不存在。



```
lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ touch check_dir.sh

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ cat > check_dir.sh
#!/bin/bash

if [ -d "$1" ]; then
    echo "Directory exists."

    count=$(find "$1" -type f | wc -l)

    echo "File count: $count"
else
    echo "Directory does not exist."
fi

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ chmod +x check_dir.sh

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ ./check_dir.sh /tmp
Directory exists.
find: '/tmp/NUW7A23.tmp.dir': Permission denied
find: '/tmp/WFP676F.tmp.dir': Permission denied
find: '/tmp/~nsul.tmp': Permission denied
File count: 2325

lx@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ ./check_dir.sh /abc
Directory does not exist.
```

图 29 目录文件数量统计脚本运行结果

2.11 文本文件单词数量统计脚本

编写 Shell 脚本 `count_words.sh`，实现输入文本文件路径并统计文件中单词数量的功能。

```
1  #!/bin/bash
2
3  file=$1
4
5  if [ -f "$file" ]; then
6      echo "File: $file"
7
8      count=$(wc -w < "$file")
9
10     echo "Word count: $count"
11 else
12     echo "File does not exist."
13 fi
```

脚本通过 `$1` 获取输入文件路径，使用 `-f` 判断文件是否存在，并利用 `wc -w` 命令统计文件中的单词数量。

测试结果表明，当输入有效文件时，脚本能够正确输出单词数量；当输入不存在的文件时，能够给出错误提示。



```
1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ touch count_words.sh

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ cat > count_words.sh
#!/bin/bash

file=$1

if [ -f "$file" ]; then
    echo "File: $file"

    count=$(wc -w < "$file")

    echo "Word count: $count"
else
    echo "File does not exist."
fi

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ chmod +x count_words.sh

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ echo "hello world shell script" > test.txt

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ ./count_words.sh test.txt
File: test.txt
Word count: 4

1x@LAPTOP-4K9EVO2A MINGW64 /d/classPai/Class/8024
$ ./count_words.sh abc.txt
File does not exist.
```

图 30 统计文本文件单词数量脚本运行结果

3 Git 与作业提交

3.1 仓库内容整理

```
8.24/  
|-- q01/  
|-- q02/  
|-- q03/  
|-- q04/  
|-- q05/  
|-- q06/  
|-- q07/  
|-- q08/  
`-- lab_report/  
    |-- figures/  
    |-- report.tex  
    |-- report.pdf  
    |-- report2.tex  
    `-- report2.pdf
```

3.2 GitHub / Gitee 提交记录

仓库地址: <https://github.com/haru-senya/8.24.git>

3.3 commit 记录截图、仓库页面截图

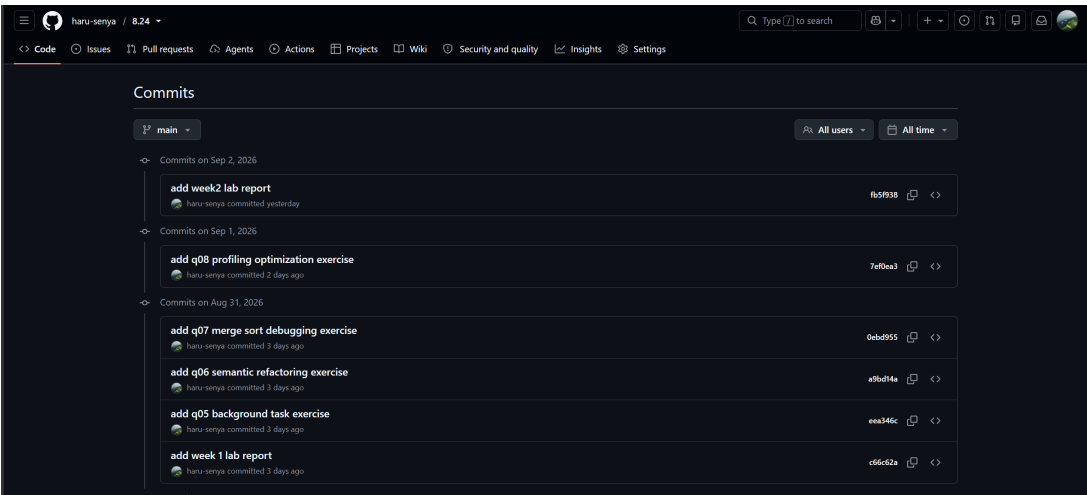


图 31 Git commit 记录

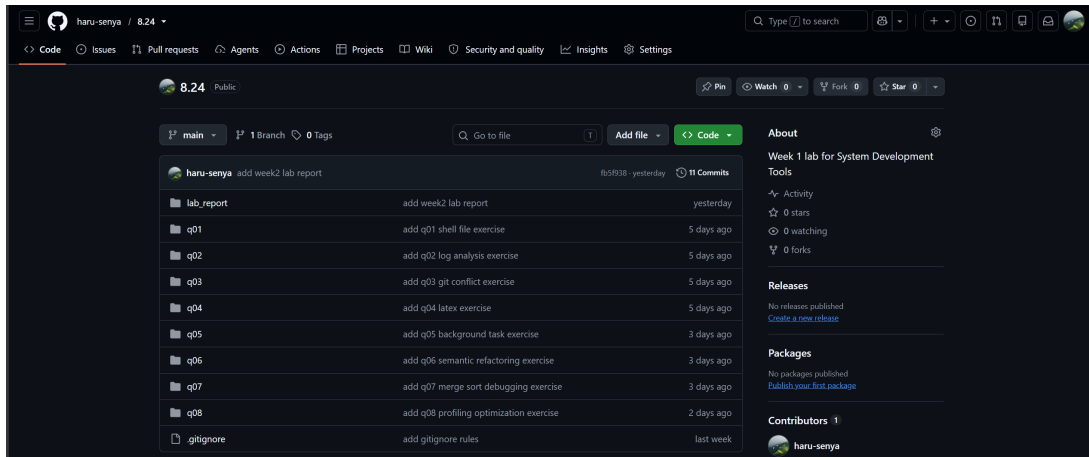


图 32 GitHub / Gitee 仓库页面

4 问题与总结

4.1 遇到的问题

- **后台任务堆积**：多次运行脚本并按 `Ctrl+Z` 挂起，终端里同时存在多个 `Stopped` 和 `Running` 任务。用 `jobs` 查看任务编号，通过 `bg` 恢复暂停任务，再用 `kill -TERM %1、%2、%3` 按作业号逐个终止。
- **不理解 `kill -TERM` 写法**：疑惑为什么不直接写 `SIGTERM`。查阅后理解：`kill` 向进程发信号，`-TERM` 是 `SIGTERM` 的简写，`$pid` 是目标进程编号。
- **手动重命名容易遗漏引用**：第 6 题把 `total_price` 改成 `calculate_total`，手动改多个文件容易漏。用 VS Code 的「重命名符号」做语义级修改，定义和所有引用同步更新，理解了语义重构和普通文本替换的区别。
- **归并排序下标用混**：程序能运行但排序结果错误。调试器观察 `i`、`j`、`left[i]`、`right[j]`，发现 `else` 分支错写 `right[i]`，应为 `right[j]`，对应的自增也应是 `j += 1`。修改后输出正确并通过测试。
- **pytest 找不到测试用例**：运行 `pytest` 显示 `collected 0 items`。检查发现测试文件命名不符合 `test_xxx.py` 规范，或测试函数没有以 `test_` 开头。修正命名后 `pytest` 正常执行。
- **set 没有 `append` 方法**：第 8 题把 `unique = []` 改 `unique = set()`，但仍写 `unique.append(word)`，报 `AttributeError: 'set' object has no attribute 'append'`。改 `unique.add(word)`，修改后运行正常，`count=1000` 不变。
- **性能优化效果验证**：用 `time` 测优化前后耗时，`cProfile` 定位瓶颈。列表换集合降低了去重的时间复杂度，性能有提升；实际耗时还受文件读取、Python 启动等因素影响，但集合在大数据量下扩展性更好。

4.2 实验总结

通过本次实验，我进一步熟悉了 Linux 环境下的进程控制、Python 开发工具使用、程序调试以及性能优化方法。在后台任务管理实验中，学习了使用 `Ctrl-Z`、`bg`、`jobs` 以及 `SIGTERM` 信号

对进程进行控制，理解了进程状态变化和信号通信机制。在 Python 工具链实验中，掌握了使用语言服务器完成代码跳转、引用查找和符号重命名，认识到语义重构相比普通文本修改更加安全可靠。

通过归并排序调试实验，我学会了使用调试器观察程序运行状态，通过断点和变量变化定位逻辑错误，而不是只依靠运行结果判断问题。在性能优化实验中，学习了使用 `time` 和 `cProfile` 对程序进行性能分析，并通过将 `list` 结构替换为 `set` 进行去重优化，理解了数据结构选择对程序效率的影响。

总体来说，本次实验不仅提升了对 Linux 工具、Git、Python 开发环境以及调试工具的使用能力，也让我认识到编写程序不仅需要实现功能，还需要关注代码质量、可维护性和运行效率。在后续学习中，需要继续加强对开发工具和工程化实践的熟练使用。